Checkpoint 3 — Projeto da hierarquia de nós da AST

Cauã Pereira Neres — 12021BCC005

1- Hierarquia de nós

Programa	Atributos	Produções de gramática do EN2 que o geram
Programa	`declaracoes: List<Declaracao>`	`Programa → DeclaracaoTopo Programa | ε`
DeclaracaoVariavel	`tipo: TipoBasico`, `nome: String`, `inicializador: Expressao?`	Global: `DeclaracaoTopo → TipoRetorno IDENT SufixoDeclaracao` com `SufixoDeclaracao → InicializacaoOpt ;`. Local: `DeclaracaoVariavel → Tipo IDENT InicializacaoOpt ;`. Em ambos, `InicializacaoOpt → = Expressao | ε`
DeclaracaoFuncao	`tipoRetorno: TipoBasico`, `nome: String`, `parametros: List<Parametro>`, `corpo: Bloco`	`DeclaracaoTopo → TipoRetorno IDENT SufixoDeclaracao` com `TipoRetorno → Tipo` e `SufixoDeclaracao → ( ParametrosOpt ) Bloco`
DeclaracaoProcedimento	`nome: String`, `parametros: List<Parametro>`, `corpo: Bloco`	Igual à anterior, mas com `TipoRetorno → void`
Parametro	`tipo: TipoBasico`, `nome: String`	`Parametro → Tipo IDENT` (dentro de `ParametrosOpt` / `ParametrosCauda`)

Bloco	itens: List<ItemBloco>	Bloco → { ItensBloco }; Comando → Bloco
Atribuicao	alvo: Identificador, valor: Expressao	Comando → IDENT SufixoComando com SufixoComando → = Expressao ;
ComandoChamada	chamada: ChamadaFuncao	Comando → IDENT SufixoComando com SufixoComando → (ArgumentosOpt) ;
Selecao	condicao: Expressao, entao: Bloco, senao: Bloco?	Comando → if (Expressao) Bloco ElseOpt; ElseOpt → else Bloco | ε
Repeticao	condicao: Expressao, corpo: Bloco	Comando → while (Expressao) Bloco
Retorno	valor: Expressao?	Comando → return ExpressaoOpt ;; ExpressaoOpt → Expressao | ε
LiteralInt	valor: int	Primaria → NUM (lexema sem ponto decimal)
LiteralDouble	valor: double	Primaria → NUM (lexema com ponto decimal, ex.: 3.14)
LiteralBool	valor: boolean	Primaria → true | false
LiteralChar	valor: char	**Sem produção no CP2** (ver observação abaixo)
LiteralString	valor: String (sem as aspas)	Primaria → STRING
Identificador	nome: String	Primaria → IDENT PrimariaSufixo com PrimariaSufixo → ε; também o alvo de uma Atribuicao

ChamadaFuncao	nome: String, argumentos: List<Expressao>	Primaria → IDENT PrimariaSufixo com PrimariaSufixo → (ArgumentosOpt); ArgumentosOpt → Expressao ArgumentosCauda | ε; também dentro de ComandoChamada
OperacaoBinaria	operador: OperadorBinario, esquerda: Expressao, direita: Expressao	Disjuncao/DisjuncaoCauda (||), Conjuncao/ConjuncaoCauda (&&), Igualdade/IgualdadeCauda (== !=), Relacional/RelacionalCauda (< > <= >=), Aditiva/AditivaCauda (+ -), Multiplicativa/MultiplicativaCauda (* / %)
OperacaoUnaria	operador: OperadorUnario, operando: Expressao	Unaria → + Unaria | - Unaria

2- Mapeamento da gramática para a árvore

Não-terminal	Destino	Justificativa
Programa	Nó Programa	A recursão DeclaracaoTopo Programa | ε é só a codificação de uma lista; vira o atributo declaracoes
DeclaracaoTopo	**Nada** (vira um dos nós de declaração)	É um ponto de decisão: conforme TipoRetorno e SufixoDeclaracao, produz DeclaracaoVariavel, DeclaracaoFuncao ou DeclaracaoProcedimento
TipoRetorno	Atributo	Vira tipo / tipoRetorno; se for void, a escolha é o nó DeclaracaoProcedimento
SufixoDeclaracao	**Nada** (fatoração)	

ParametrosOpt	Atributo	Vira `parametros`; ε vira lista vazia
ParametrosCauda	**Nada** (lista)	Só alonga a lista `parametros`; recursão de lista, não precisa de nó
Parametro	Nó `Parametro`	`Tem nome, tipo e posição próprios, e terá símbolo`
Tipo	Atributo	`Um TipoBasico (enum) dentro da declaração/parâmetro; não tem filhos, então um nó seria só um invólucro`
InicializacaoOpt	Atributo	`Vira inicializador de DeclaracaoVariavel; ε vira null`
Bloco	Nó `Bloco`	`Define escopo; é a unidade que a tabela de símbolos abre e fecha`
ItensBloco	**Nada** (lista)	`Recursão de lista; vira Bloco.itens`
ItemBloco	**Nada**	`É só uma escolha entre DeclaracaoVariavel e Comando; cada alternativa já é um nó`
DeclaracaoVariavel	Nó `DeclaracaoVariavel`	`Declara um símbolo`
Comando	**Nada** (vira um dos nós de comando)	`Escolha entre Bloco, Selecao, Repeticao, Retorno, Atribuicao, ComandoChamada`
SufixoComando	**Nada** (fatoração)	`Ver (c) abaixo`
ElseOpt	Atributo	`Vira Selecao.senao; ε vira null`
ExpressaoOpt	Atributo	`Vira Retorno.valor; ε vira null`

ArgumentosOpt	Atributo	`Vira ChamadaFuncao.argumentos; ε vira lista vazia`
ArgumentosCauda	**Nada** (lista)	`Só alonga a lista de argumentos`
Expressao	**Nada** (precedência)	`Produção unitária Expressao → Disjuncao;`
Disjuncao, Conjuncao, Igualdade, Relacional, Aditiva, Multiplicativa	**Nó `OperacaoBinaria` (só quando há operador)**	`Ver (a)`
DisjuncaoCauda, ConjuncaoCauda, IgualdadeCauda, RelacionalCauda, AditivaCauda, MultiplicativaCauda	**Nada (recursão à esquerda)**	`Ver (b)`
Unaria	**Nó `OperacaoUnaria` (só quando há +/-)**	`A alternativa Unaria → Primaria não vira nada`
Primaria	**Nada (vira literal, `Identificador` ou `ChamadaFuncao`; `( Expressao )` vira só a expressão interna)**	`Parênteses só impõem agrupamento, que a forma da árvore já representa`
PrimariaSufixo	**Nada (fatoração)**	`Ver (c) abaixo`

(a) Não-terminais criados para a precedência (como T e F)

`Disjuncao`, `Conjuncao`, `Igualdade`, `Relacional`, `Aditiva`, `Multiplicativa`, `Unaria` e a própria `Expressao` existem só para forçar, na gramática, que * ligue mais forte que +, que + ligue mais forte que <, e assim por diante. Na AST a precedência já está na forma da árvore (o operador de menor precedência fica mais perto da raiz), então os níveis somem. Em `a + b * c`, o nó * é filho do nó +, sem nenhum nó "Multiplicativa" no meio. Quando um nível não tem operador (por exemplo, `Aditiva → Multiplicativa AditivaCauda` com cauda vazia), ele não cria nada: devolve o filho como está.

(b) Não-terminais criados para eliminar a recursão à esquerda (como E' e T')

As seis ...Cauda de expressão (AditivaCauda etc.) são o resultado de transformar Aditiva → Aditiva + Multiplicativa em Aditiva → Multiplicativa AditivaCauda. Elas não têm significado próprio: são só o "resto da repetição". No parser, cada uma vira o laço while de parseAditiva e afins. A construção da AST nesse laço precisa refazer a associatividade à esquerda:

java

```java
Expressao acc = parseMultiplicativa();
while (isOneOfLexemes("+", "-")) {
    OperadorBinario op = ...;
    advance();
    Expressao dir = parseMultiplicativa();
    acc = new OperacaoBinaria(op, acc, dir, inicio.linha,
inicio.coluna);   // acc vira filho esquerdo
}
return acc;
```

Assim a - b - c vira (a - b) - c (e não a - (b - c)), mesmo a gramática sendo recursiva à direita nas caudas.

(c) Não-terminais criados pela fatoração

São os que isolam o prefixo comum de alternativas, para o parser decidir só pelo próximo token:

- SufixoDeclaracao (prefixo comum TipoRetorno IDENT): o parser lê tipo IDENT e só depois, olhando (ou =/;, decide entre DeclaracaoFuncao/DeclaracaoProcedimento e DeclaracaoVariavel.
- SufixoComando (prefixo comum IDENT): olhando = ou (, decide entre Atribuicao e ComandoChamada.
- PrimariaSufixo (prefixo comum IDENT): olhando (ou não, decide entre ChamadaFuncao e Identificador.

Nos três casos, o não-terminal é só um "ponto de decisão" sobre qual nó criar. O nó é criado depois da decisão, com o nome lido antes dela. Por isso o parser guarda o token IDENT (nome e posição) antes de decidir.

Terminais que não viram nó

- Parênteses ( ), chaves { }, vírgula , e ponto e vírgula ; são pontuação: servem para o parser reconhecer a estrutura, que a forma da árvore já carrega.

- Palavras-chave `if`, `else`, `while`, `return`, `void` e os nomes de tipo não viram nó: a escolha do tipo de nó (`Selecao`, `Repeticao`, `Retorno`, `DeclaracaoProcedimento`...) ou o valor de um enum (`TipoBasico`) já registra a informação.
- O = da atribuição/inicialização e os operadores viram atributos (a presença de `inicializador` ou de `Atribuicao`; o enum `operador`).

3- Posição

Nó	Token cuja posição o nó guarda
Programa	Primeiro token do programa (ou 1:1 se o programa for vazio)
DeclaracaoVariavel, DeclaracaoFuncao, DeclaracaoProcedimento, Parametro	O token do tipo (`int`, `void`, ...)
Bloco	`{`
Selecao	`if`
Repeticao	`while`
Retorno	`return`
Atribuicao, ComandoChamada	O `IDENT` inicial
Literais, Identificador, ChamadaFuncao	O token do literal / do `IDENT`
OperacaoUnaria	O token do operador (`+` ou `-`)
OperacaoBinaria	O **primeiro token da expressão inteira** (o início do operando esquerdo)

4- Árvore

Imagem separada para melhor visualização